Lab Report: Building AI Agents with LangGraph

Capstone Project: Intelligent University Assistant

Your Name (Achra)

November 2, 2026

Contents

1 Introduction **2**

2 Environment and Configuration **2**

3 Fundamental Concepts of LangGraph **2**

4 Tools and Security **2**

5 Capstone Project Architecture **3**
5.1 Routing Logic 3

6 Results and Evaluation **3**
6.1 Possible Improvements 3

7 Conclusion **3**

1 Introduction

The objective of this lab was to explore the **LangGraph** framework to design complex agentic workflows. Unlike traditional linear LangChain chains, LangGraph allows the creation of state graphs with loops, conditions, and memory. The highlight of this lab was designing a university assistant capable of classifying requests, using tools, and remembering the context.

2 Environment and Configuration

The project was developed in Python (version 3.13) within an isolated virtual environment. Although the original lab was designed for OpenAI, I integrated the **Google Gemini API** (model `gemini-2.5-flash`) via `ChatGoogleGenerativeAI`. The API key was managed securely using environment variables (`getpass`), avoiding any hardcoding in the source code.

3 Fundamental Concepts of LangGraph

Throughout the exercises, the following key concepts were mastered:

- **State:** A shared dictionary (e.g., `TypedDict`) that circulates between nodes. It allows nodes to read previous data and append new information.
- **Nodes:** Python functions that perform actions (e.g., calling the LLM, executing a tool) and update the state.
- **Edges:** The connections that define the execution flow. *Conditional Edges* are particularly powerful as they allow the graph to make decisions (routing).
- **Memory (Checkpointer):** Using `MemorySaver` coupled with a `thread_id` allows the agent to retain the conversation history, making it capable of remembering the user's name.

4 Tools and Security

To interact with the outside world, tools were integrated into the LLM via `bind_tools`.

- **Calculator:** The original exercise proposed using `eval()`, which represents a major security flaw. I implemented the secure version using the `ast` module (Abstract Syntax Trees) which only allows basic arithmetic operations.
- **Date/Time:** A tool to get the current date and time.

The agent is capable of autonomously deciding whether to call a tool or answer directly using its internal knowledge.

5 Capstone Project Architecture

The final project is an Intelligent University Assistant. It is a complex graph composed of:

1. A **Classifier** that analyzes the student's question.
2. Five **Specialized Agents** (Courses, Programming, Mathematics, Administration, General) dynamically generated via a *factory* function.
3. A shared **Tools Node**.
4. A **Fallback** node to handle errors (e.g., empty question).

5.1 Routing Logic

The `route_after_classifier` function inspects the category determined by the AI and directs the flow to the appropriate agent. If an agent decides to use a tool, the `route_after_agent` function redirects the flow to the tools node, and then returns to the agent to formulate the final response.

6 Results and Evaluation

During the tests (evaluation on 6 questions), the router achieved **100%** accuracy. The memory test proved the effectiveness of `MemorySaver`: the agent remembers the student's name if the conversation takes place within the same `thread_id`, but completely forgets it in another thread, thus respecting session isolation.

6.1 Possible Improvements

To make routing even more robust, the use of *Structured Outputs* (forcing the LLM to return a strict JSON format) or *Few-Shot Prompting* could reduce the risk of errors on ambiguous questions.

7 Conclusion

This lab demonstrated that LangGraph is an extremely powerful tool for going beyond the limits of simple prompt chains. The ability to create feedback loops (Agent $\rightarrow$ Tool $\rightarrow$ Agent) and manage a persistent state paves the way for much more autonomous and reliable AI applications, such as the university assistant developed in this project.